src/pages/ProductsPage.jsx JSX

```jsx
import { useEffect, useState } from 'react'
import { useSearchParams, useNavigate } from 'react-router-dom'
import { useDispatch, useSelector } from 'react-redux'
import {
  fetchProducts,
  selectProducts,
  selectProductsLoading,
  selectTotalPages,
  setFilters,
  clearFilters,
  selectFilters
} from '../store/slices/productsSlice'
import ProductCard from '../components/ProductCard'
import Skeleton from '../components/Skeleton'
import { motion } from 'framer-motion'
import SearchBar from '../components/SearchBar'
import SEO from '../components/SEO'

export default function ProductsPage() {
  const dispatch = useDispatch()
  const products = useSelector(selectProducts)
  const loading = useSelector(selectProductsLoading)
  const totalPages = useSelector(selectTotalPages)
  const filters = useSelector(selectFilters)
  const [searchParams, setSearchParams] = useSearchParams()
  const [page, setPage] = useState(1)
  const [pageSize] = useState(12)
  const [showFilters, setShowFilters] = useState(false)
  const navigate = useNavigate()

  useEffect(() => {
    // Clear any existing filters and products when component mounts
    dispatch(clearFilters())

    // Initialize filters from URL params
    const urlFilters = {
      category: searchParams.get('category') || '',
      company: searchParams.get('company') || '',
      search: searchParams.get('search') || '',
      ordering: searchParams.get('ordering') || '',
      trending: searchParams.get('trending') || ''
    }

    dispatch(setFilters(urlFilters))
  }, [searchParams, dispatch])

  useEffect(() => {
    const params = {
      ...filters,
      page,
      page_size: pageSize
    }

    // Remove empty values
    Object.keys(params).forEach(key => {
      if (!params[key]) delete params[key]
    })

    dispatch(fetchProducts(params))
  }, [filters, page, pageSize, dispatch])

  const handleSearch = (searchTerm) => {
    dispatch(setFilters({ search: searchTerm }))
    setPage(1)
  }

  const handleFilterChange = (filterType, value) => {
    dispatch(setFilters({ [filterType]: value }))
    setPage(1)
  }

  const clearAllFilters = () => {
    dispatch(clearFilters())
    setPage(1)
  }

  // Unique filter values
  const categories = [...new Set(products.map(p => p.disease_category))]
  const companies = [...new Set(products.map(p => p.company))]

  return (
    <>
      <SEO
        title="Buy Medicines Online - Best Prices & Fast Delivery | WellMed"
        description="Buy medicines online at best prices with free home delivery. Authentic medicines from top brands. Order now for fast delivery."
        keywords="buy medicine online, pharmacy, medicines, healthcare, online pharmacy, prescription drugs"
        type="website"
        link="https://wellmed.com/products"
      />
```

```jsx

      <motion.div
        initial={{ opacity: 0 }}
        animate={{ opacity: 1 }}
        className="min-h-screen bg-[#FFFDF5]"
      >
        {/* Background Pattern */}
        <div
          className="absolute inset-0 opacity-5"
          style={{
            backgroundImage: 'radial-gradient(#000 1.5px, transparent 1.5px)',
            backgroundSize: '20px 20px'
          }}
        ></div>

        {/* Header Section */}
        <div className="relative z-10 bg-[#FF6B6B] border-b-8 border-black">
          <div className="max-w-7xl mx-auto p-4 sm:p-8 text-center">
            <h1 className="text-3xl sm:text-5xl md:text-7xl font-black uppercase tracking-tight text-white mb-4 -rotate-1">
              ALL PRODUCTS
            </h1>
            <div className="bg-white border-4 border-black shadow-[6px_6px_0px_0px_#000] p-4 sm:p-6 rotate-1 inline-block mb-6">
              <p className="font-bold uppercase tracking-wide text-sm sm:text-base">
                FIND THE RIGHT MEDICINE FOR YOUR NEEDS
              </p>
            </div>

            {/* Search Bar */}
            <div className="max-w-2xl mx-auto">
              <div className="relative">
                <input
                  type="text"
                  placeholder="SEARCH FOR MEDICINES, HEALTH PRODUCTS..."
                  className="w-full border-4 border-black bg-white font-bold text-base h-14 pl-14 pr-4
                    placeholder:text-black/40 placeholder:font-bold placeholder:uppercase
                    focus-visible:bg-[#FFD93D] focus-visible:shadow-[4px_4px_0px_0px_#000]
                    focus-visible:outline-none focus-visible:ring-0
                    transition-all duration-100"
                  onKeyPress={(e) => {
                    if (e.key === 'Enter') {
                      handleSearch(e.target.value)
                    }
                  }}
                />
                <button className="absolute left-4 top-1/2 transform -translate-y-1/2">
                  <svg className="w-6 h-6 stroke-[3px]" fill="none" stroke="currentColor" viewBox="0 0 24 24">
                    <path strokeLinecap="round" strokeLinejoin="round" strokeWidth="3" d="M21 21l-6-6m2-5a7 7 0 11-14 0 7 7 0 0114 0z" />
                  </svg>
                </button>
              </div>
            </div>
          </div>
        </div>

        <div className="relative z-10 max-w-7xl mx-auto flex gap-8">
          {/* Desktop Sidebar Filters */}
          <aside className="hidden lg:block w-80 p-8">
            <div className="bg-white border-4 border-black shadow-[8px_8px_0px_0px_#000] sticky top-24 overflow-hidden">
              <div className="bg-[#FFD93D] border-b-4 border-black p-4">
                <h3 className="font-black uppercase tracking-widest text-sm">FILTERS</h3>
              </div>

              <div class="p-4 space-y-6">
                {/* Sort By */}
                <div>
                  <label className="block font-black uppercase tracking-widest text-xs mb-2">SORT BY</label>
                  <select
                    className="w-full border-4 border-black bg-white font-bold text-sm h-12 px-4
                      focus-visible:bg-[#FFD93D] focus-visible:outline-none transition-colors duration-100"
                    value={filters.ordering}
                    onChange={e => handleFilterChange('ordering', e.target.value)}
                  >
                    <option value="">DEFAULT</option>
                    <option value="mrp">PRICE: LOW TO HIGH</option>
                    <option value="-mrp">PRICE: HIGH TO LOW</option>
                    <option value="-discount">DISCOUNT: HIGH TO LOW</option>
                    <option value="name">NAME: A TO Z</option>
                  </select>
                </div>

                {/* Category */}
                <div>
                  <label className="block font-black uppercase tracking-widest text-xs mb-2">CATEGORY</label>
                  <select
                    className="w-full border-4 border-black bg-white font-bold text-sm h-12 px-4
                      focus-visible:bg-[#FFD93D] focus-visible:outline-none transition-colors duration-100"
                    value={filters.category}
                    onChange={e => handleFilterChange('category', e.target.value)}
                  >
                    <option value="">ALL CATEGORIES</option>
                    {categories.map(c => <option key={c} value={c}>{c.toUpperCase()}</option>)}
                  </select>
                </div>

```

```
                {/* Company */}
                <div>
                  <label className="block font-black uppercase tracking-widest text-xs mb-2">COMPANY</label>
                  <select
                    className="w-full border-4 border-black bg-white font-bold text-sm h-12 px-4
                      focus-visible:bg-[#FFD93D] focus-visible:outline-none transition-colors duration-100"
                    value={filters.company}
                    onChange={e => handleFilterChange('company', e.target.value)}
                  >
                    <option value="">ALL COMPANIES</option>
                    {companies.map(c => <option key={c} value={c}>{c.toUpperCase()}</option>)}
                  </select>
                </div>

                {/* Trending Toggle */}
                <div>
                  <label className="flex items-center gap-3 cursor-pointer">
                    <div className="relative">
                      <input
                        type="checkbox"
                        checked={filters.trending === 'true'}
                        onChange={e => handleFilterChange('trending', e.target.checked ? 'true' : '')}
                        className="sr-only"
                      />
                      <div className={`w-6 h-6 border-4 border-black ${
                        filters.trending === 'true' ? 'bg-[#FF6B6B]' : 'bg-white'
                      } transition-colors duration-100`}>
                        {filters.trending === 'true' && (
                          <svg className="w-4 h-4 text-white stroke-[3px] absolute inset-0" fill="none" stroke="currentColor" viewBox="0 0 24 24">
                            <path strokeLinecap="round" strokeLinejoin="round" strokeWidth="3" d="M5 13l4 4L19 7" />
                          </svg>
                        )}
                      </div>
                    </div>
                    <span className="font-bold uppercase tracking-wide text-sm">TRENDING ONLY</span>
                  </label>
                </div>

                {/* Clear Filters */}
                <button
                  className="w-full bg-white border-4 border-black font-black uppercase tracking-widest
                    text-xs py-3 h-12 shadow-[4px_4px_0px_0px_#000]
                    hover:bg-[#C4B5FD] hover:shadow-[2px_2px_0px_0px_#000] hover:translate-x-[2px] hover:translate-y-[2px]
                    active:shadow-none active:translate-x-[4px] active:translate-y-[4px]
                    transition-all duration-100"
                  onClick={clearAllFilters}
                >
                  CLEAR ALL FILTERS
                </button>
              </div>
            </div>
          </aside>

          <div className="flex-1 p-4 lg:p-8">
            {/* Mobile Filter Toggle */}
            <div className="lg:hidden mb-6">
              <button
                onClick={() => setShowFilters(!showFilters)}
                className="w-full bg-[#FFD93D] border-4 border-black font-black uppercase tracking-widest
                  text-sm py-4 h-14 shadow-[6px_6px_0px_0px_#000]
                  hover:shadow-[3px_3px_0px_0px_#000] hover:translate-x-[3px] hover:translate-y-[3px]
                  active:shadow-none active:translate-x-[6px] active:translate-y-[6px]
                  transition-all duration-100"
              >
                {showFilters ? 'HIDE FILTERS' : 'SHOW FILTERS'}
              </button>
            </div>

            {/* Mobile Filters */}
            {showFilters && (
              <div className="lg:hidden mb-6 bg-white border-4 border-black shadow-[8px_8px_0px_0px_#000] overflow-hidden">
                <div className="bg-[#C4B5FD] border-b-4 border-black p-4">
                  <h3 className="font-black uppercase tracking-widest text-sm">FILTERS</h3>
                </div>
                <div className="p-4 space-y-4">
                  <div className="grid grid-cols-2 gap-4">
                    <select
                      className="border-4 border-black bg-white font-bold text-sm h-12 px-4
                        focus-visible:bg-[#FFD93D] focus-visible:outline-none transition-colors duration-100"
                      value={filters.ordering}
                      onChange={e => handleFilterChange('ordering', e.target.value)}
                    >
                      <option value="">SORT BY</option>
                      <option value="mrp">PRICE ↑</option>
                      <option value="-mrp">PRICE ↓</option>
                      <option value="-discount">DISCOUNT ↓</option>
                    </select>
                    <select
                      className="border-4 border-black bg-white font-bold text-sm h-12 px-4
                        focus-visible:bg-[#FFD93D] focus-visible:outline-none transition-colors duration-100"
                      value={filters.category}
                      onChange={e => handleFilterChange('category', e.target.value)}
                    >
                      <option value="">CATEGORY</option>
```

```
                {categories.map(c => <option key={c} value={c}>{c.toUpperCase()}</option>)}
              </select>
            </div>
            <button
              className="bg-white border-4 border-black font-black uppercase tracking-widest
                text-xs px-4 py-2 h-10 shadow-[2px_2px_0px_0px_#000]
                hover:bg-[#C4B5FD] transition-colors duration-100"
              onClick={clearAllFilters}
            >
              RESET
            </button>
          </div>
        </div>
      )}

      {/* Results Header */}
      <div className="flex flex-col sm:flex-row justify-between items-start sm:items-center gap-4 mb-6">
        <div>
          <h2 className="text-xl sm:text-2xl md:text-3xl font-black uppercase tracking-tight">
            {filters.trending ? '🔥 TRENDING PRODUCTS' : 'ALL PRODUCTS'}
          </h2>
          <div className="bg-[#C4B5FD] border-2 border-black px-3 py-1 inline-block mt-2">
            <p className="font-bold uppercase tracking-wide text-sm">{products.length} PRODUCTS FOUND</p>
          </div>
        </div>
      </div>

      {/* Product Grid */}
      <div className="grid grid-cols-2 sm:grid-cols-2 md:grid-cols-3 lg:grid-cols-4 xl:grid-cols-5 gap-4 md:gap-6">
        {loading
          ? Array(pageSize).fill(0).map((_, i) => (
              <div key={i} className="bg-white border-4 border-black shadow-[6px_6px_0px_0px_#000] h-80"></div>
            ))
          : products.map(product =>
            <div key={product.product_id}>
              <ProductCard product={product} />
            </div>
          )
        }
      </div>

      {/* Empty State */}
      {!loading && products.length === 0 && (
        <div class="text-center py-16">
          <div class="bg-white border-4 border-black shadow-[12px_12px_0px_0px_#000] p-8 sm:p-12 inline-block -rotate-1">
            <div class="w-24 h-24 bg-[#C4B5FD] border-4 border-black mx-auto mb-6 flex items-center justify-center">
              <svg class="w-12 h-12 stroke-[4px]" fill="none" stroke="currentColor" viewBox="0 0 24 24">
                <path strokeLinecap="round" strokeLinejoin="round" strokeWidth="4" d="M21 21l-6-6m2-5a7 7 0 11-14 0 7 7 0 0114 0z" />
              </svg>
            </div>
            <h3 class="text-xl sm:text-2xl font-black uppercase tracking-tight mb-4">NO PRODUCTS FOUND</h3>
            <p class="font-bold uppercase tracking-wide mb-6">TRY ADJUSTING YOUR SEARCH OR FILTERS</p>
            <button
              onClick={clearAllFilters}
              class="bg-[#FF6B6B] border-4 border-black font-black uppercase tracking-widest
                text-sm px-6 py-4 h-14 shadow-[6px_6px_0px_0px_#000]
                hover:shadow-[3px_3px_0px_0px_#000] hover:translate-x-[3px] hover:translate-y-[3px]
                active:shadow-none active:translate-x-[6px] active:translate-y-[6px]
                transition-all duration-100"
            >
              CLEAR FILTERS
            </button>
          </div>
        </div>
      )}

      {/* Pagination */}
      {!loading && totalPages > 1 && (
        <div class="flex justify-center mt-12 gap-2">
          <button
            disabled={page === 1}
            class="px-4 py-3 h-12 border-4 border-black bg-white font-black uppercase tracking-widest text-xs
              disabled:opacity-50 disabled:cursor-not-allowed
              hover:bg-[#C4B5FD] shadow-[4px_4px_0px_0px_#000]
              hover:shadow-[2px_2px_0px_0px_#000] hover:translate-x-[2px] hover:translate-y-[2px]
              active:shadow-none active:translate-x-[4px] active:translate-y-[4px]
              transition-all duration-100"
            onClick={() => setPage(page - 1)}
          >
            PREVIOUS
          </button>

          {Array(Math.min(5, totalPages)).fill(0).map((_, i) => {
            const pageNum = i + 1;
            return (
              <button
                key={i}
                class={`w-12 h-12 border-4 border-black font-black text-sm shadow-[4px_4px_0px_0px_#000]
                  hover:shadow-[2px_2px_0px_0px_#000] hover:translate-x-[2px] hover:translate-y-[2px]
                  active:shadow-none active:translate-x-[4px] active:translate-y-[4px]
                  transition-all duration-100 ${
                  page === pageNum
                    ? 'bg-black text-white'
                    : 'bg-white hover:bg-[#FFD93D]'
```

```
                      }`}
                      onClick={() => setPage(pageNum)}
                    >
                      {pageNum}
                    </button>
                  );
                })}

                <button
                  disabled={page === totalPages}
                  className="px-4 py-3 h-12 border-4 border-black bg-white font-black uppercase tracking-widest text-xs
                    disabled:opacity-50 disabled:cursor-not-allowed
                    hover:bg-[#C4B5FD] shadow-[4px_4px_0px_0px_#000]
                    hover:shadow-[2px_2px_0px_0px_#000] hover:translate-x-[2px] hover:translate-y-[2px]
                    active:shadow-none active:translate-x-[4px] active:translate-y-[4px]
                    transition-all duration-100"
                  onClick={() => setPage(page + 1)}
                >
                  NEXT
                </button>
              </div>
            )}
          </div>
        </div>
      </motion.div>
    </>
  )
}
```